

DE 414: Practical 5 Report

Amy McDermott (26911264)

May 6, 2026

Introduction

This report details the implementation of a Convolutional Neural Network (CNN) using PyTorch. It explores defining the network architecture, the training process on the MNIST dataset, and the visualisation of the trained CNN filters.

Question 1: Loading the data

The function `load_npz` is used to import the MNIST dataset from a `.npz` file. The dataset is partitioned into training and test sets containing 60000 and 10000 samples, respectively. Each sample is a flattened 784-dimensional vector, representing a 28x28 pixel grayscale image, as verified by the dimensions shown in Listing 1.

```
1 +-----  
2 | Train X: torch.Size([60000, 784])  
3 | Train Y: torch.Size([60000])  
4 | Test X: torch.Size([10000, 784])  
5 | Test Y: torch.Size([10000])  
6 +-----
```

Listing 1: Terminal output showing the dimensions of the loaded MNIST dataset.

Question 2

Question 2.1: Defining the CNN model structure

The CNN is implemented as a subclass of `nn.Module`, with its layers initialised in the constructor. The architecture consists of:

- Convolutional layer (CNN): Extracts spatial features using a specified kernel size and stride.
- ReLU activation: Introduces non-linearity which enables the model to learn complex patterns in the data.
- Dense (fully connected) layer: Maps the flattened output from the CNN layer to the final class scores for digits 0-9.

The hyperparameters for the network are chosen to ensure that the dimensions of each layer are compatible, allowing the data to flow correctly through the network. The specific hyperparameter configurations are as follows:

- CNN layer: $P_1 = P_2 = 28$, $P_3 = 1$, $M_1 = M_2 = 8$, $M_3 = 10$, $S = 2$
- ReLU activation: $Q_1 = Q_2 = 11$, $Q_3 = 10$
- Dense layer: $P = 1210$, $Q = 10$

The code snippet in Listing 2 shows the definition of the CNN model structure in PyTorch.

```

1 self.conv_1 = nn.Conv2d(in_channels=1, out_channels=M_3, kernel_size=(M_1,
  M_2), stride=stride)
2 self.relu = nn.ReLU()
3 # Input to the linear layer is the flattened version of the conv output (i
  .e. 11 x 11 x 10 = 1210) and output is K = 10
4 self.linear = nn.Linear(in_features=1210, out_features=K)

```

Listing 2: Definition of the CNN model structure.

Question 2.2: Implementing the forward pass

The forward pass defines the transformation of input data through the network. The input vector is first reshaped to a 4D tensor to match the expected input format of the convolutional layer, which is (Batch size, Channels, Height, Width). After convolution and ReLU activation, the resulting feature maps are flattened into a 1D vector before being passed to the dense layer for classification.

```

1 # Convolutional layers expect input in the format (Batch size, Channels,
  Height, Width)
2 z_1 = self.conv_1(x.view(-1,1,28,28)) # reshape input to (N,1,28,28) for
  CNN layer
3 v_1 = self.relu(z_1)
4 # flatten the output of the convolutional layer (i.e. 11 x 11 x 10 = 1210)
5 v_1_flat = torch.flatten(v_1,start_dim=1) # flatten before passing to the
  dense layer
6 y_hat = self.linear(v_1_flat)

```

Listing 3: Implementation of the forward pass through the CNN model.

Question 2.3: Instantiating the model

The model is instantiated with a kernel size of 8, 10 output channels, and a stride of 2. We utilise the cross-entropy loss function and the SGD optimizer with a learning rate of 0.01. A batch size of 30 was selected for training over 10 epochs. The model summary is printed out to confirm the architecture and the number of parameters in each layer, as shown in the terminal output in Listing 5.

```

1 model = CNN(8,8,10,10,2) # define model
2 loss_fn = nn.CrossEntropyLoss() # define loss function
3 optimizer = torch.optim.SGD(model.parameters(), lr=lr) # define optimizer

```

Listing 4: Instantiating the CNN model and defining the loss function and optimizer.

```

1 CNN(
2   (conv_1): Conv2d(1, 10, kernel_size=(8, 8), stride=(2, 2))
3   (relu): ReLU()
4   (linear): Linear(in_features=1210, out_features=10, bias=True)

```

5)

Listing 5: Terminal output showing the model summary.

Question 3: Training the model

The training process involved 10 iterations (epochs) over the entire dataset. In each batch, the model performed a forward pass to generate predictions, followed by a backward pass to compute gradients and update weights via the optimizer. To prevent gradient accumulation across batches, `optimizer.zero_grad()` was called at the start of each step. The model's performance was monitored on both the training and test sets (Listing 8). As seen by Listing 8, the final test accuracy of 98.18% closely tracks the training accuracy of 98.53%. This narrow margin indicates that the model has successfully learned to generalize from the training data rather than simply memorizing it, suggesting that overfitting is minimal. Listing 6 and 7 shows the code snippets for training the model and the evaluation of the model on the test set, respectively.

```
1 y_hat = model(X_i)[0] # forward pass - model prediction
2 loss = loss_fn(y_hat, y_i) # calculate training loss
3
4 # clear gradients on all optimised tensors, otherwise gradients accumulate
  on subsequent backward passes
5 optimizer.zero_grad()
6 loss.backward() # calculate gradients
7 optimizer.step() # update model parameters using the calculated gradients
```

Listing 6: Model training loop, showing the process of forward pass, loss calculation, backward pass, and parameter updates.

```
1 y_hat = model(X_i)[0]
2 loss = loss_fn(y_hat, y_i)
3 # track the test loss and accuracy
4 test_loss += loss.item() * test_batch_size
5 test_accuracy += hits(y_hat, y_i)
```

Listing 7: Model evaluation on the test set during training, showing the forward pass and loss calculation for the test data.

```
1 *****
2 Beginning training
3 Epoch 1 Loss: 0.0740 Accuracy: 97.90 Test Loss: 0.0735 Test Accuracy:
  97.84
4 Epoch 2 Loss: 0.0701 Accuracy: 98.00 Test Loss: 0.0705 Test Accuracy:
  97.97
5 Epoch 3 Loss: 0.0668 Accuracy: 98.11 Test Loss: 0.0679 Test Accuracy:
  98.02
6 Epoch 4 Loss: 0.0639 Accuracy: 98.19 Test Loss: 0.0657 Test Accuracy:
  98.07
7 Epoch 5 Loss: 0.0614 Accuracy: 98.26 Test Loss: 0.0637 Test Accuracy:
  98.07
8 Epoch 6 Loss: 0.0591 Accuracy: 98.32 Test Loss: 0.0620 Test Accuracy:
  98.09
9 Epoch 7 Loss: 0.0570 Accuracy: 98.37 Test Loss: 0.0605 Test Accuracy:
  98.14
```

```

10 Epoch 8 Loss: 0.0552 Accuracy: 98.42 Test Loss: 0.0592 Test Accuracy:
    98.19
11 Epoch 9 Loss: 0.0535 Accuracy: 98.47 Test Loss: 0.0580 Test Accuracy:
    98.18
12 Epoch 10 Loss: 0.0519 Accuracy: 98.53 Test Loss: 0.0570 Test Accuracy:
    98.18

```

Listing 8: Model epoch performance during training, showing loss and accuracy for both training and test sets.

Question 4: Visualising trained CNN filters

To interpret the model’s learned behavior, a single test sample was passed through the network in evaluation mode and its corresponding filters and correlations are plotted using the `plot_filters` function. Using `torch.no_grad()`, we extracted the pre-activation maps (z_1) and post-activation feature maps (v_1). These, along with the detached convolutional weights (H), were plotted using `plot_filters`. The resulting visualisation of the trained CNN filters is shown in Figure 1.

```

1 X = test_X[0:1, :] # sample to be fed to your model
2 H = model.conv_1.weight.detach() # extract the weights from the
   convolutional layer and detach from computational graph
3 model.eval() # set model to evaluation mode
4 with torch.no_grad(): # stop tracking gradients for the forward pass
5     y_hat, z_1, v_1 = model(X) # forward pass for the sample
6
7 plot_filters(X.detach(), H, z_1.detach(), v_1.detach()) # plot the filters
   and correlations

```

Listing 9: Visualisation of the trained CNN filters.

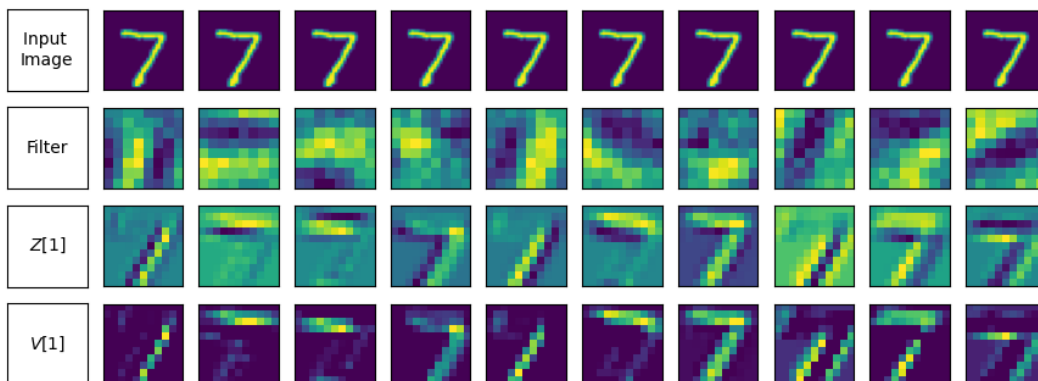


Figure 1: Visualisation of the trained CNN filters.